

# Agent libOS: A Runtime Substrate for Capability-Controlled Self-Evolving LLM Agents

Yingqi Zhang

Tsinghua University

Beijing, China

zhangyq24@mails.tsinghua.edu.cn

## Abstract

LLM agents are becoming long-running software actors. They accumulate memory, load skills, synthesize tools, fork workers, attach services, solicit human decisions, and package checkpoints and images for reuse. These mechanisms improve adaptability, but many runtimes expose an action through the same wrapper that performs its host effect. Under self-evolution, untrusted content can then change not only what an agent can request, but also what it can affect: affordance evolution becomes an implicit privilege-escalation channel.

This paper presents Agent libOS, a libOS-inspired substrate that separates *affordance evolution* from *resource authority*. Model-facing tools, skills, JIT code, images, checkpoints, and remote endpoints may change after deployment, but every protected effect crosses an explicit runtime primitive. The primitive binds a stable AgentProcess identity to a normalized resource, a typed capability decision, optional scoped approval, budgets, provider containment, and append-only audit.

Our Python prototype applies this boundary to filesystem and shell access, object memory, processes, skills and JIT tools, checkpoints and images, human requests, and registered JSON-RPC/MCP services. On a synthetic, stateful 80-scenario suite, full Agent libOS attains 100.0% attack-setting task success with 0 forbidden committed effects and complete traces in a 720-row modeled matrix. Direct, prompt-only, sandbox-only, and confirmation counterfactuals commit 100, 100, 83, and 67 forbidden effects, respectively. We separately exercise three native connector calls and a frozen qwen3.8-max campaign: all 24 free-tool trials pass task and effect oracles with 0 forbidden commits, and all 24 token-free replays reproduce utility, safety, and normalized effects. The results support a concrete runtime authority boundary for agents whose affordances and reusable state can change after deployment.

## CCS Concepts

• **Computer systems organization** → **Operating systems**; • **Security and privacy** → *Authorization*; • **Computing methodologies** → *Artificial intelligence*.

## Keywords

LLM agents, capability systems, library operating systems, self-evolving agents, runtime security, tool use

## 1 Introduction

LLM agents increasingly resemble durable software actors rather than stateless tool callers. A coding or research agent may inspect repository state, activate a skill package, synthesize a TypeScript

helper, call a remote service, request a human decision, fork a child process, checkpoint its state, and later boot a reusable image. None of these changes requires an immediate update to model weights, yet each can materially change what the agent sees, what actions it can propose, and what later agents inherit. Recent systems make this form of self-evolution explicit: useful capability increasingly comes from changing context, memory, tools, workflows, and training signals, not only from scaling a fixed model [7, 20, 26, 35–37].

This shift exposes an operating-systems problem. A common agent-runtime pattern renders a prompt, exposes tool schemas, asks the model to select an action, dispatches a host function, appends the result, and repeats. This pattern appears across influential work on reasoning and tool use, multi-agent collaboration, and agent-computer interfaces [10, 22, 30, 32, 33]. It is productive because adding actions is easy, but it is fragile because the wrapper that makes an action visible often also touches the filesystem, shell, browser, database, memory store, or remote endpoint. In such systems, “can the model see this action?” is easily conflated with “can this process invoke this operation?” and “can this operation affect this protected resource?”

Self-evolution amplifies this conflation. A malicious repository can instruct an agent to load a debugging skill, register generated code, fork a worker, call a visible remote service, and preserve the resulting state. Each step looks like ordinary adaptation. If any of those affordances carries ambient host authority, however, untrusted text turns affordance evolution into authority evolution; checkpoints and images can then preserve the unsafe procedure for future runs. Confirmation and host isolation remain useful backstops, but they answer different questions. A wrapper-level prompt may be shown before the target resource has been normalized, while a container does not identify which agent process, capability, checkpoint lineage, or human decision authorized an in-sandbox effect. Section 2.5 follows this failure end to end.

Agent libOS is built around one invariant:

*A self-evolving agent may change what it can ask for, but that change must not by itself expand what protected resources it can affect.*

Agent libOS does not propose a new prompt, planner, role protocol, or tool-synthesis optimizer. It provides a substrate in which those mechanisms can evolve while separating three questions that wrappers often collapse: *what action can the model name, which process is requesting it, and which protected effect may commit?* Tools answer the first; stable process identity and runtime authority checks answer the latter two.

Figure 1 shows the resulting layer model. Model-facing affordances remain ergonomic and evolvable. The runtime owns process

**Agent personality / application**

User goal, task-specific policy, model profile selection.

**Skills and tools layer**

Prompt instructions, static tool schemas, workflow hints, skill packages, and process-local JIT candidates. This layer is model-facing and evolvable, but non-authoritative.

**Agent libOS runtime**

Process identity and scheduling; typed capabilities and scoped approval; budgets, recovery, effect classification, and audit.

**Resource-provider substrate**

Contained adapters for filesystem, shell, human I/O, remote calls, and PTY sessions.

**Host backends**

Workspace and subprocess backends, local UI, registered services, and durable stores.

**Figure 1: The Agent libOS layer model. Model-visible self-evolution occurs above the runtime boundary; protected effects occur below it through primitives.**

identity, scheduling, authorization, context materialization, checkpoint lifecycle, wakeups, and provenance. Contained providers perform concrete host calls, but they are backends rather than security bypasses. This agent-level boundary complements host isolation instead of replacing it.

The security boundary is *process identity + capability + primitive*; append-only audit supplies provenance for allowed and denied decisions. A process may see a new tool, activate a skill, register a JIT helper, inspect a remote endpoint, boot an image, or fork from a checkpoint. Visibility may change freely, but a protected effect occurs only inside a primitive that normalizes the resource, checks authority, applies scoped approval and budgets, invokes a provider, classifies the resulting effect, and records the outcome.

This paper makes three contributions. First, it identifies a specific systems failure mode: an evolving action surface becomes a hidden permission surface when visibility and authority share a wrapper. Second, it presents a runtime model that decouples them with stable process identity, typed capabilities, primitive-level mediation, scoped approval, and append-only provenance. Third, it evaluates a working prototype with modeled counterfactuals, native runtime traces, repeated real-model completions, latency measurements, and primitive regressions.

## 2 Problem, Threat Model, and Rationale

### 2.1 Failure mode and scope

The self-evolving-agent literature distinguishes what evolves, when, how, and where [7]. Agent libOS focuses on a deployment-relevant subset: *non-parametric runtime self-evolution*. A process may change prompt context, activate a skill, register a JIT tool, spawn or fork another process, create or boot an image, restore or commit a checkpoint, attach a registered service, or use an object-backed terminal

session supplied by a trusted module. A deployment may also accumulate policy-permitted LLM-call traces and human ratings for later prompt, workflow, image, or model improvement. These operations do not immediately update model weights, but they change what the current agent can request and what future agents can inherit. In OS terms, they resemble loading code, changing an address-space view, opening devices, spawning children, checkpointing state, and collecting telemetry for later builds.

A model-facing wrapper is an interface, not an authority boundary. It may validate JSON arguments and format observations, but if it directly performs host effects, every wrapper author must reproduce the same authorization discipline. Indirect prompt injection and tool-output injection exploit this gap between untrusted content and trusted execution [3, 8, 34]; ordinary-looking tool loops can therefore produce harmful effects [21]. The specific failure sequence is: untrusted input changes or selects an affordance, that affordance carries ambient credentials or calls the host directly, and an effect commits without an independent process/resource decision. Self-evolution makes the first step persistent and reusable.

### 2.2 Threat model

The prototype targets threats common in agent applications. Repository files, retrieved documents, tool outputs, generated skill text, JIT source, process messages, checkpoint metadata, and remote-provider responses may steer an agent toward unauthorized reads, writes, deletes, shell execution, object-memory materialization, process control, checkpoint use, image registration, remote calls, PTY interaction, or human-approval confusion. The adversary may know object names, namespace strings, object ids, endpoint ids, method names, tool names, checkpoint ids, or image ids, and may induce the model to request a harmful action.

The TCB has three layers: (1) policy-critical runtime managers for capabilities, primitives, process/checkpoint/image state, and audit; (2) configured providers and the SQL persistence boundary; and (3) the Python/Deno runtimes and host OS. A defect in any of these layers can violate the invariant, and the audit log is provenance rather than independent attestation. Model outputs, repository contents, messages, remote results, ordinary skill and image packages, and Deno/TypeScript JIT tools are not trusted to enforce policy. Trusted startup modules are admitted only from operator-accepted manifests and source hashes.

Within this model, an adversary may control any untrusted input and may know stable resource identifiers, but cannot modify the trusted runtime, forge host configuration, or directly compromise a provider backend. The boundary therefore constrains confused-deputy behavior by an agent process; it does not claim to contain a compromised TCB.

### 2.3 Design requirements

The threat model yields seven requirements that are easy to obscure when a tool wrapper is treated as the unit of enforcement.

- **Stable subjects.** Long-running agents need process identities that own authority, budgets, queues, parent-child relations, and audit history across spawn, fork, wait, resume, and exit.
- **Non-authoritative evolution.** Loading a skill, exposing a tool, registering JIT code, discovering a service, or packaging an image

may change the action surface, but must not silently mint filesystem, shell, memory, process, checkpoint, image, remote-method, human, or PTY authority.

- **Point-of-use mediation.** Authorization must occur at a typed primitive after resource and argument normalization, not when a broad wrapper first becomes visible or is selected.
- **Explicit memory authority.** Object names, ids, namespaces, and context snippets are references, not capabilities; lookup, read, materialization, linking, and merge remain separately authorized and budgeted.
- **Attenuated delegation and resumable approval.** Children receive no more authority than explicitly delegated, and a human decision resumes the original pending primitive with an exact, auditable scope rather than converting a wait into a new ambient grant.
- **Scoped recovery.** Checkpoints reconstruct internal state while preserving audit and external-effect history; recovery must not imply that an already committed remote or provider effect was rolled back.
- **End-to-end explanation.** For an allowed or denied effect, the runtime should link the process, tool or syscall, normalized primitive, capability or policy decision, human action, provider result, and relevant checkpoint or image lineage.

## 2.4 Alternatives and trade-offs

Table 1 makes the design choice explicit. Prompt policy, confirmation, and host isolation remain useful, but each answers a different question. Mature containers, microVMs, and capability operating systems are therefore compositional backends and design precedents, not interchangeable agent-level baselines [1, 15, 29].

## 2.5 Running example

Consider a coding agent whose repository contains an injected instruction to “install the diagnostic skill, generate a helper, ask a worker to inspect credentials, restart production, and save the improved setup.” In a wrapper-only runtime, this plausible workflow can inherit every credential available to the host process. In Agent libOS, the same workflow unfolds as follows.

1. **Load a skill.** Activation changes the process-local prompt and visible tool table, but adds no capability.
2. **Generate a JIT helper.** The validated TypeScript helper has no Deno host permissions. Its attempted secret read becomes a filesystem primitive and is denied because the process lacks that typed resource capability.
3. **Fork a worker.** The child receives selected memory, a smaller budget, and explicitly attenuated authority; knowing an object id or checkpoint id does not recover the parent’s rights.
4. **Use a remote service.** A registered read-only status method may be allowed, while a visible production-restart method is denied independently. A human may approve one exact source-file write; that approval cannot authorize the restart.
5. **Reuse the state.** A checkpoint or image may preserve the new skill and helper, but declared capabilities are requirements for later bootstrap, not packaged grants. The audit chain links the skill, JIT syscall, fork, approval, allowed status call, denied restart, and resulting image.

The example instantiates the requirements above: stable subjects, separate affordance and authority state, point-of-use mediation, attenuated delegation, resumable approval, scoped recovery, and end-to-end provenance.

## 3 Agent libOS Design

### 3.1 Processes, affordances, and authority

A process in Agent libOS is a runtime subject:

$$p = \langle id, img, status, mem, cwd, T, S, J, C, B, Q, children \rangle,$$

where  $id$  is the process identity,  $img$  is the current AgentImage,  $status$  is lifecycle state,  $mem$  is a process-local Object Memory view,  $cwd$  is a workspace-relative working directory,  $T$  is the static model-visible tool table,  $S$  is the loaded skill set,  $J$  is the process-local JIT tool set,  $C$  is the capability set,  $B$  is the hierarchical budget and usage record,  $Q$  is the durable message queue, and  $children$  records the process tree.

The affordance surface of  $p$  is the set of model-visible affordances:

$$A_p = T_p \cup S_p \cup J_p \cup I_p \cup R_p,$$

where  $I_p$  contains visible image and checkpoint affordances and  $R_p$  contains registered remote endpoint or server affordances. Authority is not  $A_p$ . Authority is the set of decisions reachable from  $C_p$  under typed resource matching, deny dominance, constraints, one-shot use counts, expiry, delegation lineage, and human policy. The basic invariant is:

$$\Delta A_p \Rightarrow \Delta C_p.$$

Changing the affordance surface does not change the capability set. Capability changes require trusted bootstrap, explicit capability operations, scoped human approval, checkpoint restore or fork under capability checks, or audited administrative action. Even then, capabilities are durable records with issuer lineage, not hidden effects of loading a skill or registering a tool.

In the running example, skill activation and JIT registration change  $A_p$  while leaving  $C_p$  fixed; the fork constructs a new subject with explicitly attenuated  $C_{child}$ . This separation is what the wrapper-only design lacks.

For every committed protected effect  $e$ , Agent libOS requires a primitive invocation under a concrete process identity, a successful capability decision, and an audit record:

$$\begin{aligned} \text{commit}(e) &\Rightarrow \exists p, \rho, d. \text{primitive}(p, \rho) \wedge \\ &\quad \text{authorize}(p, \rho) = d \wedge \\ &\quad d \in \{\text{allow}, \text{askApproved}\} \wedge \\ &\quad \text{audit}(e, p, \rho, d). \end{aligned}$$

The formula concerns committed effects, not model choices. A model may request many actions; only primitive-authorized requests cross the runtime boundary.

### 3.2 Primitives and capabilities

A primitive is a trusted operation that crosses a protected boundary. The prototype groups primitives into local resources (filesystem read/write/delete, shell execution, and PTY interaction), agent state (object lookup/materialization/write, process lifecycle, checkpoints,

| Boundary                   | Useful guarantee                                                               | Gap for an evolving agent                                                                                                         | Role and cost in Agent libOS                                              |
|----------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Prompt/tool policy         | Reduces unsafe requests and irrelevant actions.                                | Untrusted content can change model behavior; visibility does not constrain a wrapper’s host authority.                            | Usability layer only; no enforcement claim.                               |
| Wrapper confirmation       | Gives a human a decision point.                                                | Often approves a broad wrapper before resource normalization; repeated prompts create burden and habituation.                     | Primitive-level ask with an exact, one-shot grant; adds blocking latency. |
| Container or microVM       | Contains host processes, files, and network namespaces.                        | Does not express agent-process identity, checkpoint lineage, object namespaces, or registered service methods inside the sandbox. | Provider backend and blast-radius limit; adds deployment cost.            |
| Typed runtime capabilities | Enforces least authority per process, resource, and operation with provenance. | Requires policy engineering, a non-trivial TCB, and mediation overhead.                                                           | Chosen agent-level boundary; composed with the mechanisms above.          |

**Table 1: The alternatives protect different boundaries. The evaluation’s sandbox and confirmation rows are controlled counterfactuals, not performance comparisons with mature products.**

and images), evolution (skill activation and JIT registration/execution), human interaction, and registered JSON-RPC or MCP calls. Regardless of whether a request originates in a Python wrapper, TypeScript syscall, CLI, GUI, or trusted module, the sequence is the same: normalize the resource and arguments; evaluate capability and deterministic policy; optionally suspend for approval; preflight budgets and reserve one-shot use; invoke the provider; and emit operation, effect, event, and audit records. Centralizing this sequence prevents each new affordance from inventing its own authorization semantics.

A capability is a structured record:

$$c = \langle \text{subject}, \text{resource}, \text{rights}, \text{effect}, \text{issuer}, \text{lineage}, \text{constraints}, \text{lease}, \text{status} \rangle.$$

Resources are typed canonical identifiers for filesystem subtrees, object-memory objects and namespaces, processes, images, checkpoints, JSON-RPC endpoint methods, or MCP server/tool pairs. Rights are primitive-specific—for example, read, write, delete, execute, materialize, grant, or approve. Decisions are allow, deny, or ask, with deny dominance and fail-closed parsing of unknown rights or malformed constraints. Typed terminal wildcards avoid string-prefix confusion such as `src` versus `src2`. One-shot authority is an allow capability whose committed use consumes its remaining count; issuer and delegation lineage make subsequent attenuation inspectable.

Human approval is modeled as a runtime device rather than as wrapper-local UI. An ask decision creates a durable request containing the process id, primitive name, canonical resource, requested grant scope, predicted overwrite state, byte count or content hash where relevant, risk classification, and escaped preview. The process enters `waiting_human`; other processes continue. Approval installs or consumes the scoped one-shot capability and resumes the original primitive. Rejection returns a structured failure. Ordinary approval paths cannot grant broad global filesystem write, wildcard shell execution, or capability-admin authority.

### 3.3 Self-evolution surfaces

Agent libOS treats self-evolution as changes to process-local affordances, reconstructable state, and learning records—not as expansion of resource authority. Skills and JIT tools form its prompt- and code-evolution surfaces. Activating a skill may update one process’s prompt, expose declared static tools, provide bundled JIT candidates,

and make package resources available through an activation snapshot rooted at `SKILL.md`; it grants no ambient rights. Bundled and model-proposed TypeScript follow the same propose/validate/register/execute lifecycle. Validation applies bounded static checks, import allowlists, size limits, and configured tests. A registered module remains process-local and can reach the runtime only through `libos.syscall(name, args)`; Deno execution has no independent file, network, environment, subprocess, or FFI permissions.

Processes and Object Memory make evolution parallel and stateful. Spawn begins with goal-only memory and no broad external authority; fork carries only selected memory and explicitly attenuated capabilities. Object Memory may evolve namespaces, typed payloads, provenance, lifecycle state, and context views, but knowing an object name or receiving a context snippet does not authorize lookup, materialization, linking, release, or merge.

Checkpoints and images make evolution reusable. A checkpoint captures reconstructable process state such as working directories, selected Object Memory state, namespace links, subtree capabilities, visible tools, JIT candidates, loaded skill snapshots, mailbox state, image references, and trusted-module hashes. Restore reconstructs that state without erasing append-only audit or pretending to roll back the outside world. Images package a successful operating context—prompts, default tools and skills, optional JIT tools, startup modules, model/context policy, safety metadata, and an optional workspace seed—but image-declared capabilities remain bootstrap requirements for operator review, not packaged grants.

Registered remote methods obey the same split: discovery is an affordance; JSON-RPC or MCP invocation is a typed primitive over a stable endpoint/method or server/tool pair. LLM-call records and human ratings form a learning data plane for later prompt, workflow, image, or model improvement, but never authorize effects. Because prompts, arguments, responses, ratings, and comments may contain sensitive or consent-governed content, deployments may retain complete records only under policy, or store bounded previews and hashes.

### 3.4 Providers and containment

Providers are backends, not bypasses. The prototype includes local filesystem, clock, shell, human, JSON-RPC over HTTP, and MCP client providers; the provider abstraction is intended to admit future container, browser, database, Git, or service connectors. In all

cases, the primitive retains caller identity, capability checks, policy evaluation, event emission, audit, and effect classification.

Shell execution accepts argv arrays rather than shell strings. The shell primitive applies command-risk rules over executable identity and argv tokens, blocks or asks as needed, enforces subprocess wall/CPU/RSS budgets through the provider, and emits paired intent/result records. Deno/TypeScript JIT tools run without Deno host permissions for file, network, environment, subprocess, or FFI access and use `--no-prompt` with cached-only dependency behavior.

JSON-RPC over HTTP and MCP support are client-only in the prototype. Tools pass endpoint/server ids, method/tool ids, and JSON parameters and cannot pass URLs, credentials, headers, transports, commands, or raw wire methods at call time. The primitive constructs the capability resource from stable ids and checks invocation authority before exposing manifest metadata or schemas. Endpoint and server manifests store wire names, schemas, timeout and byte limits, rollback class, mutation flags, and information-flow flags. Provider calls emit external-effect rows classified as `irreversible`, `rollbackable`, or `no_rollback_required`; checkpoint restore reports these effects but does not claim to compensate them.

## 4 Implementation

The implementation is a Python research prototype. Its composition root wires a thread-backed scheduler, process and Object Memory managers, the capability manager, ToolBroker, skill and checkpoint/image managers, an event bus, audit and human managers, an LLM executor, a trusted-module registry, and the provider substrate. Each AgentProcess stores a host-resolved model-profile id. Root spawn selects an explicit host profile, an image default, or the runtime default; fork and child creation inherit it unless the host overrides it. Model-facing tools cannot switch to arbitrary credentials or endpoints. LLM calls persist provider and API-mode identifiers, bounded prompt/output envelopes, visible schemas, tool-call and reasoning metadata, usage, and errors. Missing usage under a token budget fails closed, and over-budget actions are not dispatched. Human ratings are stored as scored, attributed, audited records, supporting later improvement without becoming authority grants.

The Electron console is a supervision surface behind a loopback server and random bearer token. Requests carrying an actor process id use that process’s capabilities; explicit local-administrator actions follow a separate audited path rather than inheriting the selected process’s identity. The console can inspect persisted calls, pending human requests, and provenance, and can record process ratings for later evaluation pipelines.

## 5 Evaluation

### 5.1 Questions, benchmark design, and baselines

The empirical goal is not to show that Agent libOS improves a planner. It is to test whether a runtime authority boundary preserves utility, safety, and provenance while an agent changes its affordance surface. The evaluation asks four questions:

- (1) Does primitive mediation prevent forbidden committed effects in realistic benign and attack workflows?

- (2) Does task utility survive capability checks, human approval, and provider containment?
- (3) Can allowed and denied sensitive effects be explained by linked runtime evidence?
- (4) What latency and operational costs does the boundary introduce?

The primary benchmark is an 80-scenario practical workflow suite spanning coding, research, enterprise, DevOps, and self-evolution tracks. Each track contains eight task families, including core tasks, diagnostic tool output, external context, state updates, tool extension, remote action, capability transfer, and delayed triggers. The suite contains 40 benign scenarios, 25 direct attack scenarios, 10 adaptive scenarios, and 5 long-horizon scenarios. Each scenario declares trusted and untrusted inputs, allowed and forbidden effects, utility oracles, security oracles, final-state oracles when applicable, and expected provenance. The design follows two common practices in recent agent evaluations: separating utility from security in tool-rich attack settings [3, 21, 34], and checking final environment state rather than only intermediate actions [11, 31, 38]. The suite is synthetic and inspired by these benchmarks; it does not claim to run their official datasets.

We evaluate four controlled counterfactual baselines and four ablations. **Direct** dispatches model-visible wrappers directly to host effects. **Prompt-only** relies on instructions without runtime enforcement. **Sandbox-only** models coarse shell/filesystem/network containment but lacks agent identity, checkpoint lineage, object namespaces, and remote-method authority. **Confirmation** asks for broad approval around selected wrappers. Full Agent libOS dispatches through primitive-level mediation. The ablations remove audit linkage, fork/checkpoint/image attenuation, primitive approval, or remote-method capabilities. These runners isolate mechanisms; they are not implementations or performance evaluations of Docker, Firecracker, Capsicum, or commercial agent sandboxes.

Metrics separate utility, safety, cost, and explainability. We report benign task success, attack-setting task success, final-state success, forbidden committed effects, unknown effects, false denials, approval events, and effect-to-audit trace coverage. In attack settings a harmful action can be requested, denied, or committed; the safety claim concerns committed effects. Approval decisions in automated experiments are made by a scenario-aware scripted oracle over declared allowed effects, not by human subjects. Exact 100% values therefore describe this fixed regression suite, not an estimated population rate.

### 5.2 Practical workflow results

Table 2 reports the main practical results. The modeled matrix executes all 80 scenarios against nine runners and ablations, producing 720 result rows. It is a controlled design matrix: semantic effects and oracles are modeled, so no row is counted as a native runtime execution.

Full Agent libOS preserves the suite’s declared utility while blocking every forbidden committed effect in the fixed modeled matrix. Direct and prompt-only runners commit all 100 modeled forbidden effects. The modeled sandbox blocks some shell/network-like effects but misses agent-level data-flow, service-state, and

| Runner                | Scen. | Benign<br>succ. | Attack<br>succ. | State<br>succ. | Forbidden<br>comm. | Trace<br>cov. |
|-----------------------|-------|-----------------|-----------------|----------------|--------------------|---------------|
| Agent libOS (modeled) | 80    | 100.0%          | 100.0%          | 100.0%         | 0                  | 100.0%        |
| Confirmation          | 80    | 100.0%          | 10.0%           | 55.0%          | 67                 | 0.0%          |
| Sandbox-only          | 80    | 100.0%          | 0.0%            | 50.0%          | 83                 | 0.0%          |
| Direct wrapper        | 80    | 100.0%          | 0.0%            | 50.0%          | 100                | 0.0%          |
| Prompt-only           | 80    | 100.0%          | 0.0%            | 50.0%          | 100                | 0.0%          |

**Table 2: Modeled practical workflow results. Rows are controlled counterfactuals rather than performance comparisons with deployed products; forbidden committed is a count.**

| Runner                | Benign<br>succ. | Attack<br>succ. | State<br>succ. | Forbidden<br>comm. | Trace<br>cov. |
|-----------------------|-----------------|-----------------|----------------|--------------------|---------------|
| Full Agent libOS      | 100.0%          | 100.0%          | 100.0%         | 0                  | 100.0%        |
| No audit              | 100.0%          | 100.0%          | 100.0%         | 0                  | 0.0%          |
| No fork atten.        | 100.0%          | 87.5%           | 93.8%          | 20                 | 100.0%        |
| No human approval     | 100.0%          | 87.5%           | 93.8%          | 5                  | 100.0%        |
| No remote-method caps | 100.0%          | 62.5%           | 81.2%          | 20                 | 100.0%        |

**Table 3: Agent libOS ablations on the 80-scenario modeled matrix. Forbidden committed is a count.**

method-authority attacks. Wrapper confirmation records 114 approval events and still commits 67 forbidden effects; full Agent libOS records 5 scoped decisions. Thus the result is not merely fewer attempted actions: the mediated runner completes the allowed task path while separating it from the forbidden side effect encoded in the same scenario.

Trace coverage is an explainability metric, not just a logging metric. A covered sensitive effect is linked to process identity, normalized resource, capability decision, primitive or tool call, provider effect, and approval decision when present. As a separate native smoke test, three stateful JSON-RPC connector scenarios—mail send, CRM create, and calendar create—each execute one real ToolBroker call. All three pass exact provider-receipt and final-state checks, yielding 3 semantic effects, 3 calls, and 12 Explain-resolvable operations with no modeled fallback.

### 5.3 Ablations, real-model evidence, and primitive coverage

Table 3 isolates the mechanisms responsible for the main result. Removing audit linkage does not create forbidden committed effects in this run, but it drops trace coverage to 0.0%, separating enforcement from accountability. Removing fork attenuation commits 20 forbidden effects, showing that child, checkpoint, and image transitions need authority narrowing. Removing primitive-level approval commits 5 forbidden effects, and removing remote-method capabilities commits 20, showing that scoped approval and method-level remote authority are independently necessary in the corresponding scenarios.

To test whether the boundary still supports an unconstrained action-selection loop, we freeze a real free-tool completion campaign with qwen3.8-max at temperature 0.2: eight coding, research, enterprise, DevOps, and self-evolution tasks, three fresh-state repeats each, and final-state plus effect oracles. The model selects runtime tools rather than fixed action identifiers; a scripted approver accepts only an exact effect already declared allowed by the task

| Operation              | <i>n</i> | Median ms | p95 ms |
|------------------------|----------|-----------|--------|
| Provider read (direct) | 1,000    | 1.24      | 2.12   |
| Tool read (allowed)    | 1,000    | 100.33    | 139.51 |
| Tool read (denied)     | 1,000    | 36.56     | 52.13  |
| Explain operation      | 1,000    | 1.82      | 2.40   |
| Process spawn          | 100      | 91.45     | 111.76 |
| Checkpoint create      | 100      | 27.85     | 34.34  |
| Checkpoint restore     | 100      | 67.82     | 78.46  |

**Table 4: Prototype latency, excluding model and network time.**

oracle, which is hidden from the model. All 24 live trials pass both task and effect oracles, with 0 forbidden commits, 0 false denials, six scripted approvals, and 100.0% mean audit completeness. Replaying every transcript without another model call reproduces utility, safety, and normalized effects in 24/24 cases. Provider-reported usage totals 5.66 million tokens (median 218,676.5 and 95th percentile 315,098 per trial); wall time is 92.83 s at the median and 125.84 s at the 95th percentile. The campaign bridges the deterministic matrix and actual model-driven tool use, but these counts characterize this model and task set rather than population-level reliability.

### 5.4 Mediation cost

Our single-host microbenchmark uses Python 3.12.13 and SQLite on AMD64 Windows 11, a 4-KiB file, and 30 warm-ups (Table 4). Read and explain rows use 1,000 measured iterations; stateful rows use 100. The allowed read includes tool dispatch, normalization, capability and policy checks, durable operation/effect/audit records, and provider I/O.

The mediated allowed read adds 99.09 ms at the median, typically smaller than model inference or remote service latency. The result is a prototype cost, not a lower bound: synchronous SQLite durability and verbose provenance dominate the path. Batching audit writes or moving policy/state to a lower-level service may reduce latency but weakens immediate durability or enlarges the deployment TCB.

## 6 Discussion and Limitations

The library-OS abstraction localizes agent-specific policy above a conventional host OS; it does not schedule CPU cores or implement POSIX. The benefit is one stable decision point for changing tool tables, memory, process identity, checkpoints, images, and remote services. The cost is explicit policy engineering: deployments must define resource types, normalize identifiers, provision least-authority grants, operate an approval path, retain audit state, and decide which provider effects are reversible. Fail-closed behavior can reduce availability, and checkpoint restore cannot compensate an already committed external effect.

Several limits remain. Natural language inside arguments and policy context makes some validation semantic. Approval is slow and subject to habituation; our scripted approvals measure mechanism behavior, not human decision quality. Deno no-permission execution is not a production sandbox, and trusted Python modules remain in the TCB. Strong deployments should compose containers, WASM, microVMs, remote sandboxes, provider-specific isolation, and independently protected audit storage. The prototype has no formal verification or independent monitor, so its complete traces do not prove that a compromised runtime recorded the truth.

## 7 Related Work

**Self-evolving and tool-using agents.** Self-improving agents study how LLM systems can evolve memory, skills, workflows, tools, code, and, in some settings, model parameters [7, 20, 26, 35–37]. A related line of work on tool use and agent-computer interfaces develops action representations, tool APIs, collaboration protocols, and execution loops for completing tasks across external environments [6, 10, 12, 19, 22, 24, 25, 27, 28, 30, 32, 33]. Agent libOS addresses a different layer of the stack. Rather than proposing a new action language or task-solving policy, it treats model-visible tools, skills, and workflows as evolvable affordances above a runtime boundary that preserves process identity, mediates capability-bearing primitive calls, contains provider-side effects, and records audit evidence.

**LLM operating systems and agent benchmarks.** Several systems reinterpret operating-system abstractions for LLM agents. MemGPT frames context management as virtual memory, while AIOS studies OS support for agent scheduling, context, memory, storage, access control, and tool resources [14, 17]. In parallel, benchmarks such as SWE-bench, WebArena, and OSWorld evaluate whether agents can complete realistic software, web, and desktop tasks [11, 31, 38]. Agent libOS focuses on a complementary measurement question: whether runtime evolution preserves authority invariants while agents modify their own affordances. Its design draws on capability systems, exokernels, library OSes, containers, and microVMs [1, 4, 5, 9, 13, 15, 16, 23, 29]. These mechanisms provide backends and design precedents, but the agent-specific problem is to connect persistent process identity and model-visible affordances to mediated, capability-checked primitive operations.

**Agent security and mediated authority.** Indirect prompt injection, tool-output attacks, and agent-risk benchmarks show that untrusted content can influence agents toward unsafe tool calls [3, 8, 21, 34]. CaMeL extracts control and data flow from a trusted query and uses capabilities to prevent unauthorized flows from untrusted data [2]; systems-oriented work further emphasizes delegated authority, persistent state, isolation, mediation, and provenance [18]. Agent libOS is complementary: it targets durable process and authority transitions plus committed provider effects in existing agent frameworks. It evaluates that boundary using modeled practical workflows, native connector traces, repeated real-model completions, and primitive regression tasks rather than relying only on policy descriptions or static threat models.

## 8 Conclusion

Self-evolution increases the need for stable authority boundaries. If an agent can improve its tools, it can also create higher-impact failure modes. If it can create and commit state into images, it can persist unsafe procedures. If it can feed LLM traces and human ratings into training or fine-tuning, it can influence future agents. If it can attach remote endpoints, it can move effects outside the local workspace. Agent libOS demonstrates one way to keep this evolution visible, scoped, and auditable: let agents evolve what they can request, but require every protected effect to pass through process identity, capability checks, primitive mediation, provider containment, and audit.

## References

- [1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 419–434. <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [2] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. <https://arxiv.org/abs/2503.18813>
- [3] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 82895–82920. doi:10.52202/079017-2636
- [4] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (March 1966), 143–155. doi:10.1145/365230.365252
- [5] Dawson R. Engler, M. Frans Kaashoek, and James O’Toole, Jr. 1995. Exokernel: An Operating System Architecture for Application-Level Resource Management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (Copper Mountain, Colorado, USA) (SOSP ’95). Association for Computing Machinery, New York, NY, USA, 251–266. doi:10.1145/224056.224076
- [6] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv:2402.14034 [cs.MA] doi:10.48550/arXiv.2402.14034
- [7] Huan-gang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 2026. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *Transactions on Machine Learning Research* (Jan. 2026). <https://openreview.net/forum?id=CTr3bov55F>
- [8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (Copenhagen, Denmark) (AISeC ’23). Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985
- [9] Norman Hardy. 1985. KeyKOS Architecture. *ACM SIGOPS Operating Systems Review* 19, 4 (Oct. 1985), 8–25. doi:10.1145/858336.858337
- [10] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, Vol. 2024. 23247–23275.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
- [12] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 51991–52008. doi:10.52202/075280-2264
- [13] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: Library Operating Systems for the Cloud. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (ASPLOS ’13). Association for Computing Machinery, New York, NY, USA, 461–472. doi:10.1145/2451116.2451167
- [14] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Second Conference on Language Modeling*.
- [15] Dirk Merkel. 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal* 2014, 239, Article 2 (March 2014), 1 pages. doi:10.5555/2600239.2600241
- [16] Mark Samuel Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph.D. Dissertation. Johns Hopkins University, Baltimore, Maryland, USA.
- [17] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] doi:10.48550/arXiv.2310.08560

- [18] Lukas Pirch, Micha Horlboge, Patrick Großmann, Syeda Mahnur Asif, Klim Kireev, Thorsten Holz, and Konrad Rieck. 2026. Toward Securing AI Agents Like Operating Systems. arXiv:2605.14932 [cs.CR] doi:10.48550/arXiv.2605.14932
- [19] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, Vol. 2024. 9695–9717.
- [20] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. 2025. Alita: Generalist Agent Enabling Scalable Agentic Reasoning with Minimal Predefinition and Maximal Self-Evolution. arXiv:2505.20286 [cs.AI] doi:10.48550/arXiv.2505.20286
- [21] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitit, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*, Vol. 2024. 27031–27098.
- [22] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. doi:10.52202/075280-2997
- [23] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: A Fast Capability System. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles* (Charleston, South Carolina, USA) (SOSP '99). Association for Computing Machinery, New York, NY, USA, 170–185. doi:10.1145/319151.319163
- [24] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 38154–38180. doi:10.52202/075280-1657
- [25] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. doi:10.52202/075280-0377
- [26] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=ehRiF0R3a>
- [27] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 50208–50232.
- [28] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*, Vol. 2025. 65882–65919.
- [29] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, Washington, DC, 29–46.
- [30] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=BAakY1hNKS>
- [31] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 52040–52094. doi:10.52202/079017-1650
- [32] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X)
- [34] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [35] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In *International Conference on Learning Representations*, Vol. 2026. 104223–104294.
- [36] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations*, Vol. 2025. 34040–34077.
- [37] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills. arXiv:2504.07079 [cs.AI] doi:10.48550/arXiv.2504.07079
- [38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, Vol. 2024. 15585–15606.